Artificial Intelligence

Mid Syllabus

THEORY PART (Solutions)

Theory questions related to **AI mid-syllabus**
(containing 6 - 8 marks)

These questions and solutions are exclusively prepared by
Mohammad Abu Obaida Mullick,
ID: 011212057

Informed & Uninformed Search Algorithm

i. Uniform cost search is the same as Breadth first search if all step costs are equal.

No, uniform cost search and breadth-first search are not the same, even if all step costs are equal. While both algorithms are uninformed search strategies, they have different exploration approaches and priorities.

The main difference between BFS and UCS lies in how they prioritize node expansion –
BFS focuses on exploring nodes at the same depth level first, while UCS prioritizes nodes with the lowest cumulative path cost.

If all step costs are equal, UCS becomes identical to BFS in terms of exploration order, but their fundamental strategies remain distinct.

---

ii. Greedy best-first search is optimal when the heuristic function is admissible.

Yes, that's correct. Greedy Best-First Search (GBFS) is an informed search algorithm that uses a heuristic function to guide its search towards the goal. In GBFS, the node with the lowest heuristic value is expanded first.

If a heuristic value is equal to its actual value, there will be no need to expand any additional nodes to reach the goal node. An admissible heuristic which is less than the actual cost, always expands less number of additional nodes possible to reach the goal node. The fewer the additional nodes expand, the more the search is optimal.

It is important to note that while GBFS is optimal with an admissible heuristic, it might not be complete, meaning it might not find a solution if one exists. This is because GBFS does not consider the actual path cost

and might get stuck in cycles or dead ends.

---

iii. Greedy Best First search with consistent heuristics will not always return the optimal solution.

Yes. Greedy Best-First Search (GBFS) with consistent (also known as monotonic or non-decreasing) heuristics does not guarantee an optimal solution.

Greedy Best-First Search simply focuses on expanding the node that appears to be closest to the goal based on the heuristic value. A consistent heuristic can guide the search effectively and often lead to good solutions, but it is not enough to ensure optimality. GBFS with consistent heuristics might get stuck in local optima, where it chooses a path that looks promising based on the heuristic but later finds out that it is not the optimal path.

iv. Depth Limited Search is complete only if l >= s (Here, l=depth limit, s=depth of shallowest solution).

That statement is incorrect. Depth-limited search (DLS) is never fully complete, even if the depth limit (l) is greater than or equal to the depth of the shallowest solution (s).

The completeness of a search algorithm refers to its ability to find a solution if one exists. DLS explores the tree by recursively traversing each branch until the specified depth limit is reached. In this case, it is not complete because the solution might lie beyond the depth limit **l**, and DLS will not explore deeper levels to find it. It could terminate prematurely without finding a solution even if one exists at a deeper level.

***Extra:
The completeness condition for DLS should be **l >= d,** where d is the depth of the search space. If l is greater than or equal to the maximum depth of the search space, then DLS is complete because it will explore the entire search space and eventually find the solution if it exists.

---

v. Iterative Deepening Search is optimal if all edge costs are equal.

Yes, Iterative Deepening Search (IDS) is an optimal search algorithm when all edge costs are equal.

Iterative Deepening Search is a combination of Depth-First Search (DFS) and Breadth-First Search (BFS) strategies. It repeatedly applies DFS with increasing depth limits until a solution is found. With each iteration, the depth limit is increased by one.

If all edge costs are equal, then the depth of the shallowest solution (s) equals the cost of the optimal solution. In this case, since IDS explores the search space in increasing depth levels, it will reach the shallowest solution first. Therefore, IDS is guaranteed to find the optimal solution when all edge costs are equal.

---

vi. A* search algorithm using the heuristic h(n) = c for some fixed constant c > 0 is guaranteed to find an optimal solution.

No, using the heuristic h(n) = c for some fixed constant c > 0 in the A* search algorithm does not guarantee finding an optimal solution. In fact, this type of heuristic is not admissible, and A* may not produce the optimal path to the goal.

To be an admissible heuristic, h(n) must satisfy the following condition for all nodes n in the search space:
$h(n) \leq c(n)$, where c(n) is the actual cost from node n to the goal. In other words, the heuristic value must never overestimate the true cost to reach the goal.

In the case of h(n) = c for some fixed constant c > 0, the heuristic is not admissible because it is a constant value that does not depend on the state of the node or the problem domain. When A* uses this non-admissible heuristic, it might

expand nodes in suboptimal order, potentially leading to a non-optimal solution. The algorithm may still find a solution, but it is not guaranteed to be the best or optimal one.

vii. Greedy Best-First search using an admissible heuristic is guaranteed to find an optimal solution.
No, Greedy Best-First Search (GBFS) using an admissible heuristic is not guaranteed to find an optimal solution.

While GBFS with an admissible heuristic is an informed search algorithm that uses the heuristic value to guide its search towards the goal, it does not provide an optimality guarantee. The main reason for this is that GBFS focuses solely on the heuristic value at each step and does not consider the actual path cost to reach a node.

An admissible heuristic is one that never overestimates the true cost to reach the goal from a given node. While an admissible heuristic is important for guiding the search efficiently and towards promising paths, it does not guarantee that GBFS will always find the optimal solution. In some cases, GBFS might get stuck in a local minimum, where it chooses a path that appears promising based on the heuristic but does not consider the actual path cost, leading to a suboptimal solution.

---

viii. "Iterative Deepening DFS is not optimal when all the step costs are not identical" – draw a state-space graph and corresponding search tree that supports this statement.

Since the IDS-DFS explores nodes in increasing depth levels, it first finds the path with fewer steps (depth) to the goal. However, it does not guarantee that the path found will have the lowest total cost.

ix. Suppose that for some specific problem, heuristic function h is admissible. Is $h(n)^2$ admissible? How about $\sqrt{h}$? If your answer is yes for any of these two functions, would you prefer them over h? Explain your answer.

Given that, h(n) is admissible.

$h(n)^2$ can be 3 different types of admissible:

1. **h(n) is more than 1, (can overshoot the actual value!)**
For example,
h(n) = 3, actual value = 5
but $h(n)^2$ = 9 (Not admissible)

2. **h(n) is equal to 1(Remains same!)**
no change!
h(n) = 1, actual value = 1.5
but $h(n)^2$ = 1 (Both are admissible and non-dominating)

3. **h(n) is less than 1(Weakly admissible)**
can be less than h(n)
for example,
h(n) = 0.75, actual value = 1
but $h(n)^2$ = 0.5625 (admissible but weak)

$\sqrt{h(n)}$ is Weakly admissible!
For example,
h(n) = 9, actual value = 10
but $\sqrt{h(n)}$ = 3 (admissible but weak)

So, it depends on the heuristic value while choosing these new heuristic values, I would prefer the h(n) over these as the admissibility of $h(n)^2$ varies and $\sqrt{h(n)}$ is always weakly admissible.

xi. Breadth First Search is a special case of the Uniform Cost Search algorithm.

No, Breadth-First Search (BFS) and Uniform Cost Search (UCS) are two distinct search algorithms, and BFS is not a special case of UCS. While both are uninformed search strategies, they have different exploration approaches and priorities.

While it is true that BFS and UCS may produce the same result if all step costs are equal, they are fundamentally different algorithms. BFS expands nodes based on their depth levels, while UCS considers the actual path cost to guide its search.

In summary, BFS and UCS are separate search algorithms, and although they may behave similarly under certain circumstances, they have distinct strategies and are not considered special cases of each other.

xii. Uniform Cost Search is a special case of the A* Search algorithm.

A* Search is a more general algorithm that can be adapted to use different heuristic functions for informed search. Uniform Cost Search is a special case of A* Search when the heuristic function h(n) is set to zero, and it becomes an uninformed search algorithm.

When h(n) = 0, A* Search and UCS behave identically, expanding nodes based solely on the actual path cost.

Local Search

1. Define the Random Restart Hill Climbing search algorithm. What are the advantages over the greedy version?

Random Restart Hill Climbing (RRHC) is a local search algorithm used for optimization problems. It is a variant of the Hill Climbing algorithm with the addition of random restarts to escape local maxima.

Advantages of the Greedy Version:

- Escaping Local Maxima: Random Restart Hill Climbing is less likely to get stuck in local maxima. The random restarts allow the algorithm to explore different areas of the search space, increasing the chances of finding a global optimum instead of just settling for a local maxima.
- Increased Solution Quality: The random restarts in RRHC help improve the quality of the solution by allowing the algorithm to iteratively refine and explore different parts of the search space.

Random Restart Hill Climbing is more likely to find a better solution than the one found by the simple greedy Hill Climbing algorithm.

---

2. **If the temperature decreases in simulated annealing, how does the probability of bad moves change?**

In the simulated annealing algorithm, as the temperature decreases, the probability of accepting bad moves also decreases.

Temp increases, the probability increases

As the temperature decreases over time (annealing process), the algorithm becomes more selective and less willing to accept worse solutions. The probability of accepting bad moves diminishes, and the algorithm becomes more focused on exploiting and refining the best solutions found so far.

The purpose of this temperature-driven acceptance of worse solutions is to enable the simulated annealing algorithm to escape local optima and explore other parts of the search space. As the temperature decreases, the

algorithm transitions from a more exploratory mode to a more exploitative mode, aiming to converge to a better and possibly optimal solution.

In summary, as the temperature decreases in simulated annealing, the probability of accepting bad moves decreases, leading the algorithm to be more selective and focused on finding better solutions while reducing the chances of getting trapped in local optima.

3. **If the temperature is zero all the time in simulated annealing, how will the search behave? Which version of greedy search will it resemble then? Explain your answer.**

If the temperature is kept at zero all the time in simulated annealing, the search behavior will essentially become equivalent to a greedy local search algorithm also known as Hill Climbing Algorithm.

In this scenario, if T = 0, the algorithm then will not give any chances to the worst node, as a result, an optimal result may not come. The algorithm will become deterministic and will always choose the best available option at each step, without any probabilistic exploration. It will behave like Hill Climbing Algorithm, selecting the best neighboring state at each iteration, even if it leads to a slightly worse solution than the current one.

---

4. **The mutation step in the genetic algorithm increases diversification – Explain.**

The mutation step increases diversification in a genetic algorithm:

- **Exploring New Solutions:** The mutation introduces random changes to individuals, which may lead to entirely new solutions not present in the original population.
- **Escaping Local Optima:** Without mutation, the genetic algorithm might get stuck in local optima, where the population converges to a suboptimal solution.
- **Maintaining Genetic Diversity:** Over time, through selection and crossover, the genetic algorithm might lose genetic diversity as the population converges toward promising solutions.

However, it is essential to strike a balance with the mutation rate. If the mutation rate is too high, the genetic algorithm might introduce too much randomness, hindering the convergence to better solutions.

5. **Suppose you are trying to solve the 8 queens problem using greedy hill-climbing search. How will the performance be affected if you allow sideways move (choose equal-valued successors if better ones are not generated)?**

Allowing sideways moves in the greedy hill-climbing search can improve the performance of the algorithm for certain problem instances, including the 8 queens problem. By allowing sideways moves, the algorithm can escape local optima and explore a wider region of the search space, potentially finding better solutions and avoiding getting stuck in suboptimal plateaus.

The performance of the greedy hill-climbing search with sideways moves will be affected in the following ways:

- Improved Exploration: Allowing sideways moves increases the exploration capability of the search algorithm. It can navigate through plateaus and explore different paths to find better solutions potentially.
- Potential for Better Solutions: By continuing the search beyond local optima, the algorithm has a higher chance of reaching a better solution that might have been missed without allowing sideways moves.

- Longer Search Time: Allowing sideways moves can prolong the search process since the algorithm continues exploring even when it reaches a plateau. As a result, the search might take longer to converge to a solution.

- Not Guaranteed to Find the Optimal Solution: While sideways moves can enhance exploration, the greedy hill-climbing search with sideways moves is not guaranteed to find the optimal solution for complex problems like the 8 queens problem. It is still a local search algorithm and can get stuck in certain cases.

In conclusion, allowing sideways moves in the greedy hill-climbing search can be a valuable addition, particularly for problems with complex and rugged search spaces, like the 8 queens problem. It can improve the chances of finding better solutions and enhance the algorithm's overall performance. However, it may still have limitations and not guarantee the optimal solution in all cases.

6. **What is the difference between Randomly Restarting Hill Climbing Search k times and local beam search with a beam size of k?**

Randomly Restarting Hill Climbing Search k times:
A variant of the HCA that addresses the problem of getting stuck in local optima. It involves running the HCA multiple times from different random starting points (random restarts) and selecting the best solution found among all the attempts.

Key features:

- It performs the basic Hill Climbing algorithm k times with different random initial states.
- After each iteration, the algorithm records the best solution found so far.
- It is suitable for non-greedy optimization problems with complex and rugged search landscapes where the choice of the initial state significantly impacts the outcome.
- It aims to overcome the issue of local optima by diversifying the search across multiple starting points.

Local Beam Search with a beam size of k:
A population-based search algorithm that maintains multiple states (solutions) concurrently and explores new states by generating successor states. The beam size k determines how many states are retained at each step. It selects the k best states based on some evaluation criterion and continues the search from these selected states.

Key features:

- It maintains a population of k states (solutions) at each iteration.
- The algorithm generates successor states for all k states and selects the best k states from the generated successors based on an evaluation criterion (e.g., objective function value).
- It performs a form of parallel search where multiple states are explored and maintained simultaneously.
- Local Beam Search is suitable for problems where the initial state is not as critical as in Hill Climbing, and the performance depends more on exploring the search space efficiently.

Main Differences:

- **Approach:** Randomly Restarting Hill Climbing performs the basic Hill Climbing algorithm k times from different random starting points. In contrast, Local Beam Search maintains a population of k states and performs parallel searches with a selection process to retain the best k states.

- **State Retention:** In Randomly Restarting Hill Climbing, the algorithm retains the best solution found among the k attempts. In Local Beam Search, the algorithm retains k states throughout the search process.

- **Diversification vs. Intensification:** Randomly Restarting Hill Climbing focuses on diversifying the search by trying different starting points, aiming to escape local optima. Local Beam Search focuses on intensification by maintaining a set of promising solutions and exploring their successors to find the best solution.

- **Search Landscape:** Randomly Restarting Hill Climbing is beneficial for complex search landscapes with multiple local optima. Local Beam Search is suitable for problems with relatively smoother search landscapes, where the initial starting point may not significantly impact the outcome.

7. **What will happen in the local beam search if we set the beam size(k) to be 1?**

By keeping only one state in the search, the algorithm essentially becomes a basic hill-climbing algorithm. It explores the search space by iteratively moving to the best neighboring state, aiming to improve the objective function value (maximize or minimize) based on the evaluation criterion.

However, it's important to note that with a beam size of 1, the Local Beam Search loses the ability to diversify the search and explore multiple states simultaneously. This can be a limitation in exploring large and complex search spaces, as it may get easily stuck in local optima and have limited global search capabilities.

---

8. **What is the role of the temperature, T in Simulated Annealing? How do the chances of taking bad moves change with the value of T?**

Simulated Annealing is a probabilistic optimization algorithm. The temperature (T) controls the acceptance of "bad moves." At high temperatures, it explores widely, and at low temperatures, it converges to better solutions. Proper tuning of T is crucial for a successful search.

---

9. **A greedy hill-climbing search has very low memory requirements.**

Greedy hill-climbing search has very low memory requirements as it only maintains the current state and its best neighboring state during the search. It does not keep a history of previously visited states, resulting in minimal memory usage. However, this lack of memory can lead to a myopic search, where the algorithm gets stuck in local optima and fails to backtrack or explore alternative paths. While memory-efficient, it may sacrifice global exploration and hinder finding the optimal solution, especially in complex and rugged search spaces.

PEAS Specifications

1. Suppose, you are going to design an intelligent AI gardener that can water your rooftop plants and clear the weeds. The AI gardener needs to water the plants in the right amount without wetting the rooftop. It also needs to identify and then pluck out the weeds when necessary. Determine the PEAS specification for the agent. Characterize the agent's environment as Deterministic vs. Stochastic and Static vs. Dynamic.

PEAS specification for the intelligent AI gardener:

Performance measures: plant growth, overall plant health, weed-free rooftop, water consumption efficiency, and the avoidance of water spillage on the rooftop.

Environment: The rooftop garden where the plants are located.

Actuators: A watering system, a robotic arm with gripper, etc.

Sensors: Soil moisture sensors, Plant health sensors, Weed detection sensors, Weather sensors, etc.

Characterization of the agent's environment is Stochastic and static.

Stochastic: While the environment is mostly deterministic, there can still be some stochastic elements. For example, factors like weather conditions (e.g., rain, wind) can introduce some level of uncertainty in the gardening process. Additionally, there might be unexpected events or changes that can affect the garden's condition.

2. Suppose you are designing an artificially intelligent Medical Diagnosis System. The system is designed to receive information like symptoms, medical history, etc. from the patient and doctor, and after an interactive Q and A session, the system can provide a possible diagnosis for the patient. Determine the PEAS specification for the agent and also characterize the agent's environment as static vs. dynamic and discrete vs. continuous.

PEAS specification for the intelligent AI gardener:

Performance measures: plant growth, overall plant health, weed-free rooftop, water consumption efficiency, and the avoidance of water spillage on the rooftop.

Environment: The rooftop garden where the plants are located.

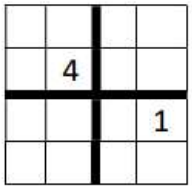
Actuators: A watering system, a robotic arm with gripper, etc.

Sensors: Soil moisture sensors, Plant health sensors, Weed detection sensors, Weather sensors, etc.

Characterization of the agent's environment is Stochastic and static.

---

3. Suppose, you are going to design an intelligent AI tennis-playing agent that can play tennis against an opponent on the other side of the court. The agent can track the opponent's movement and the ball's progress and can respond accordingly. Determine the PEAS specification for the agent. Characterize the agent's environment as Static vs. Dynamic and Discrete vs. Continuous.



The PEAS specification for the AI tennis-playing agent:

Performance Measure: Winning the game, successful shots, accuracy, and points earned.

Environment: Dynamic and Continuous (due to the changing positions of the opponent and the ball).

Actuators: Motors for movement and an arm or racket to hit the ball.

Sensors: Cameras or vision-based sensors to track the opponent and the ball, and other sensors to detect the agent's position and racket orientation.

Characterization of the agent's environment is dynamic and continuous.

Formal Description

1. Following is a 4x4 sudoku. The goal is to fill the 4X4 grids with numbers so that each row, column, and 2X2 section contains all of the digits between 1 and 4. Give a formal description of this problem as

a search problem.

Formal Description of the Sudoku 4x4 Problem as a Search Problem:

State Space: The total boxes are 4X4 = 16 - 2 (2 boxes are already given) = 14,
There are 4 inputs (1, 2, 3, 4),
The total number of steps can be 4^{14}.

Initial State: The initial state is a partially filled 4x4 Sudoku grid where two cells already have numbers 1 and 4 filled in, and other cells are empty.

Actions: Place a valid number (1 to 4) in an empty cell of the grid and no row, column, or 2x2 section contains duplicate numbers.

Transition Function: When the agent chooses to place a number in an empty cell, the corresponding cell in the grid is filled with the chosen number. If the number violates any of the Sudoku rules (e.g., duplicate numbers in the same row, column, or 2x2 section), the action is not allowed, and the state remains unchanged.

Goal Test: If each row, column, and 2x2 section contains all the digits from 1 to 4 without any duplicates.

Loop(i, j):
Are boxes[i][j] != duplicate numbers?

2. Suppose there is a 4X4 grid of squares and three of these squares contain a prize. You are designing an AI agent who will start at the southwest corner of the grid and navigate through the squares until it is able to obtain all three prizes. Give a formal description of this problem as a search problem. What is the size of the state space?

Formal Description of the Problem as a Search Problem:

State Space: There are three prizes, each with two possible states (either present or not present), and the agent can be in any of the 16 squares. Hence, the state space size is $^{16}C_3 \times n^2 \times 2^3$.

Initial State: The southwest corner of the 4x4 grid.

Actions: Up, Down, Left, Right

Transition Function: The transition model defines the result of an action taken by the agent. If the agent takes a valid action, it moves to the neighboring square in the corresponding direction.

Goal State: The goal state is reached when the agent has obtained all three prizes, i.e., it is located in a square with a prize. The agent must visit each of the three prize squares to achieve the goal.

Cost:

3. Suppose you are designing an artificially intelligent wall painting robot. The wall is divided into 144 squares in a 12 x 12 grid design. The robot can paint a single square in a singular action. It then moves the hand to the next square and paints. The job is finished when the whole wall is painted. Give a formal description of this problem as a search problem. What is the size of the state space?

Formal Description of the Problem as a Search Problem:

State Space: All the not-painted square walls

Initial State: The initial state is the robot located at the starting position, with no squares painted on the wall.

Actions: The robot can paint a single square on the wall at a time. It can move its hand to the neighboring square horizontally or vertically to reach the next unpainted square.

Transition Model: The transition model defines the result of an action taken by the robot. If the robot paints a square and moves to a neighboring square, the new state will reflect the updated wall with the newly painted square.

Goal State: The goal state is all 144 squares are filled with paint.

Cost: Assuming all actions have the same cost (uniform cost), the cost of each action is constant.

Made with ❤️ by Mohammad Abu Obaida Mullick